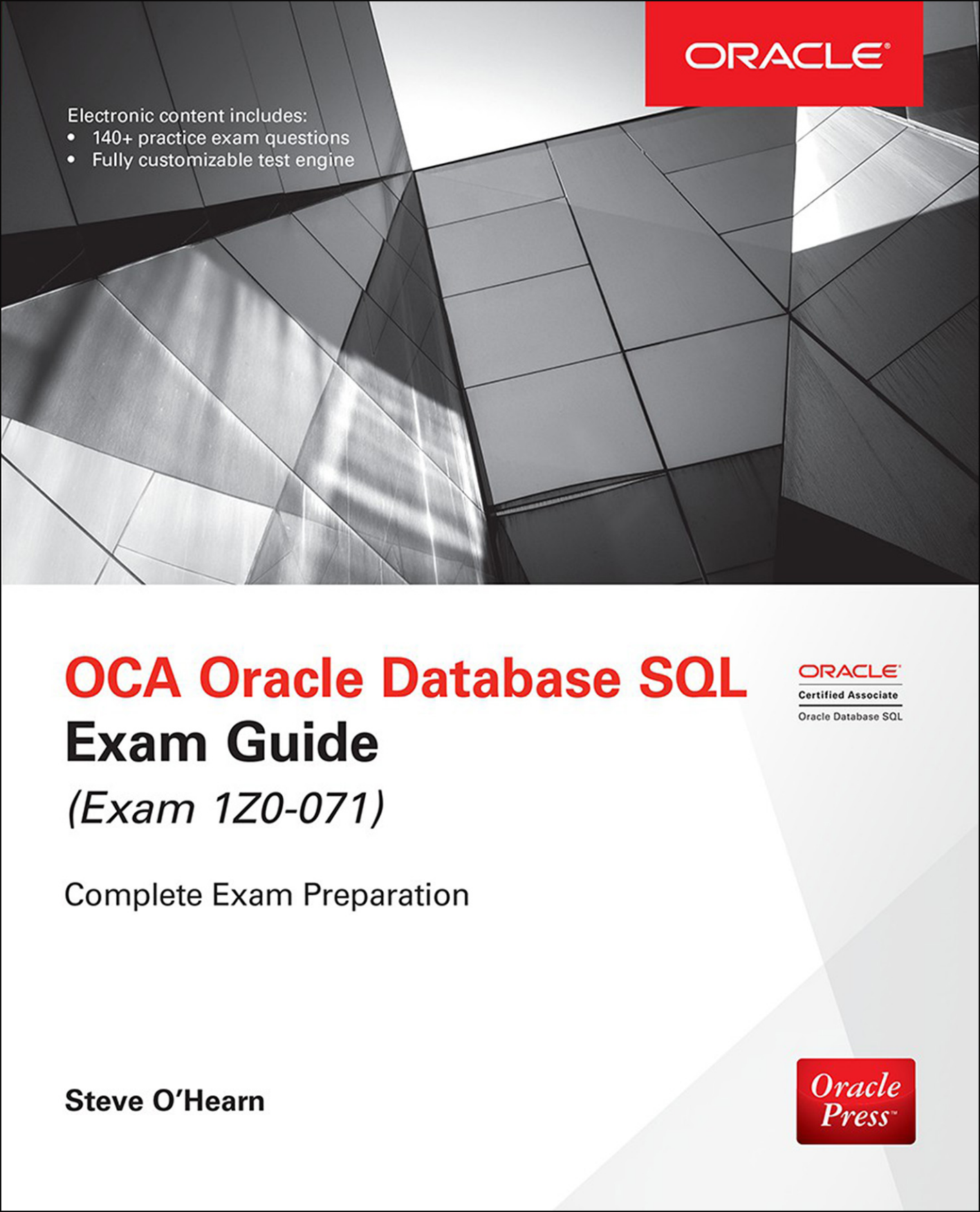




ORACLE®

Electronic content includes:

- 140+ practice exam questions
- Fully customizable test engine

OCA Oracle Database SQL

Exam Guide

(Exam 1Z0-071)

ORACLE®
Certified Associate
Oracle Database SQL

Complete Exam Preparation

Steve O'Hearn



Oracle
Press™

ORACLE®

Oracle Press™

OCA Oracle Database SQL Exam Guide

(Exam 1Z0-071)

This page intentionally left blank

The Oracle logo, consisting of the word "ORACLE" in a white, sans-serif, uppercase font, followed by a registered trademark symbol (®), all contained within a solid black rectangular background.

Oracle Press™

OCA Oracle Database SQL Exam Guide

(Exam 1Z0-071)

Steve O'Hearn

McGraw-Hill Education is an independent entity from Oracle Corporation and is not affiliated with Oracle Corporation in any manner. This publication and digital content may be used in assisting students to prepare for the OCA Oracle Database SQL exam. Neither Oracle Corporation nor McGraw-Hill Education warrants that use of this publication or digital content will ensure passing the relevant exam. Oracle® and Java™ are registered trademarks of Oracle and/or its affiliates. Other names may be trademarks of their respective owners.



New York Chicago San Francisco
Athens London Madrid Mexico City
Milan New Delhi Singapore Sydney Toronto

Copyright © 2017 by McGraw-Hill Education. All rights reserved. Except as permitted under the United States Copyright Act of 1976, no part of this publication may be reproduced or distributed in any form or by any means, or stored in a database or retrieval system, without the prior written permission of the publisher, with the exception that the program listings may be entered, stored, and executed in a computer system, but they may not be reproduced for publication.

ISBN: 978-1-25-958461-9
MHID: 1-25-958461-5.

The material in this eBook also appears in the print version of this title: ISBN: 978-1-25-958549-4,
MHID: 1-25-958549-2.

eBook conversion by codeMantra
Version 1.0

All trademarks are trademarks of their respective owners. Rather than put a trademark symbol after every occurrence of a trademarked name, we use names in an editorial fashion only, and to the benefit of the trademark owner, with no intention of infringement of the trademark. Where such designations appear in this book, they have been printed with initial caps.

McGraw-Hill Education eBooks are available at special quantity discounts to use as premiums and sales promotions or for use in corporate training programs. To contact a representative, please visit the Contact Us page at www.mhprofessional.com.

Oracle is a registered trademark of Oracle Corporation and/or its affiliates. All other trademarks are the property of their respective owners, and McGraw-Hill Education makes no claim of ownership by the mention of products that contain these marks.

Screen displays of copyrighted Oracle software programs have been reproduced herein with the permission of Oracle Corporation and/or its affiliates.

Oracle Corporation does not make any representations or warranties as to the accuracy, adequacy, or completeness of any information contained in this Work, and is not responsible for any errors or omissions.

TERMS OF USE

This is a copyrighted work and McGraw-Hill Education and its licensors reserve all rights in and to the work. Use of this work is subject to these terms. Except as permitted under the Copyright Act of 1976 and the right to store and retrieve one copy of the work, you may not decompile, disassemble, reverse engineer, reproduce, modify, create derivative works based upon, transmit, distribute, disseminate, sell, publish or sublicense the work or any part of it without McGraw-Hill Education's prior consent. You may use the work for your own noncommercial and personal use; any other use of the work is strictly prohibited. Your right to use the work may be terminated if you fail to comply with these terms.

THE WORK IS PROVIDED "AS IS." McGRAW-HILL EDUCATION AND ITS LICENSORS MAKE NO GUARANTEES OR WARRANTIES AS TO THE ACCURACY, ADEQUACY OR COMPLETENESS OF OR RESULTS TO BE OBTAINED FROM USING THE WORK, INCLUDING ANY INFORMATION THAT CAN BE ACCESSED THROUGH THE WORK VIA HYPERLINK OR OTHERWISE, AND EXPRESSLY DISCLAIM ANY WARRANTY, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO IMPLIED WARRANTIES OF MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE. McGraw-Hill Education and its licensors do not warrant or guarantee that the functions contained in the work will meet your requirements or that its operation will be uninterrupted or error free. Neither McGraw-Hill Education nor its licensors shall be liable to you or anyone else for any inaccuracy, error or omission, regardless of cause, in the work or for any damages resulting therefrom. McGraw-Hill Education has no responsibility for the content of any information accessed through the work. Under no circumstances shall McGraw-Hill Education and/or its licensors be liable for any indirect, incidental, special, punitive, consequential or similar damages that result from the use of or inability to use the work, even if any of them has been advised of the possibility of such damages. This limitation of liability shall apply to any claim or cause whatsoever whether such claim or cause arises in contract, tort or otherwise.

*To Jim Bauchspies, who gave me my first Oracle job
and who has always been like a second father to me.*

This page intentionally left blank

ABOUT THE AUTHOR

Steve O'Hearn is a technology consultant with expertise in the design, development, and administration of various data and data-driven systems for such clients as the U.S. Defense Department, NASA HQ, the FAA, the World Bank, and many others. He first became an Oracle Certified Professional (OCP) in 2001 and is a certified Oracle Database SQL Expert. He has a degree in business administration with a specialization in information processing from George Washington University, and his postgraduate work includes advanced statistics and the completion of the Future of e-Government Executive Education training at Harvard University's Kennedy School of Government in 2003. He is a member of Mensa.

Mr. O'Hearn authored *OCE Oracle Database SQL Certified Expert Exam Guide (Exam 1Z0-047)*, the official certification exam guide for Oracle's first expert-level certification exam, and the critically acclaimed *OCP Developer: PL/SQL Program Units Exam Guide*, both from Oracle Press. He contributed to *Oracle Web Applications 101* from Oracle Press and *Oracle8 Server Unleashed*. He has been featured on Oracle TV and Oracle Open World/JavaOne. He has been published in a variety of publications, including the *Harvard Business Review* and the *Record* of the National Press Club, where he is an officially recognized subject-matter expert on the topics of database and information technology. He has been vice president and conference coordinator for the Mid-Atlantic Association of Oracle Professionals. He leads the official Java User Group (JUG) for the Baltimore Washington Metropolitan Area, possesses certifications in Java and other technologies, and loves working with his own Raspberry Pi and 3D printing devices.

Mr. O'Hearn provides Oracle and Java technology training and tutoring online. He blogs at Skere9.com. He invites any and all inquiries at soh@corbinian.com.

About the Contributor

Bob Bryla is an Oracle 9i, 10g, 11g, and 12c Certified Professional with more than 20 years of experience in database design, database application development, training, and Oracle database administration. He is the primary Oracle DBA and database scientist at Epic in Verona, Wisconsin. He is a technical editor for

a number of Oracle Press books, including several certification study guides for Oracle Database 10g, 11g, and 12c. You can also find him on Lynda.com teaching Oracle Database fundamentals in bite-sized videos. He has also been known to watch science-fiction movies, tinker with Android devices, and refinish antique furniture in his spare time.

About the Technical Editor

Todd Meister has been working in the IT industry for more than 20 years. He's been a technical editor on more than 75 books ranging in topic from SQL Server to the .NET Framework. Besides technical editing, he is the assistant vice president/ chief enterprise architect at Ball State University in Muncie, Indiana. He lives in central Indiana with his wife, Kimberly, and their five bright children.

CONTENTS AT A GLANCE

1	Oracle and Structured Query Language (SQL)	1
2	Using DDL Statements to Create and Manage Tables	41
3	Manipulating Data	105
4	Restricting and Sorting Data	147
5	Using Single-Row Functions to Customize Output	201
6	Using Conversion Functions and Conditional Expressions	245
7	Reporting Aggregated Data Using the Group Functions	279
8	Displaying Data from Multiple Tables	319
9	Using Subqueries to Solve Queries	351
10	Managing Schema Objects	389
11	Using the Set Operators	449
12	Managing Objects with Data Dictionary Views	473
13	Manipulating Large Data Sets	497
14	Controlling User Access	529
A	About the Download	565
	Glossary	569
	Index	587

This page intentionally left blank

CONTENTS

<i>Acknowledgments</i>	<i>xxi</i>
<i>Preface</i>	<i>xxv</i>
<i>Introduction</i>	<i>xxxi</i>
<i>Exam Readiness Checklist</i>	<i>xxxiii</i>
1 Oracle and Structured Query Language (SQL)	1
The Exam: An Overview	2
What to Do and What to Expect	4
Oracle SQL vs. ANSI SQL	7
Oracle SQL vs. Oracle SQL*Plus	7
SQL Fundamentals I vs. SQL Certified Associate	7
Confirm Appropriate Materials for Study	13
Identify the Connection Between an ERD and a Relational Database	16
Entity-Relationship Diagrams and Data Modeling	17
Relational Databases	20
Many-to-Many Relationships	22
Database Normalization	23
Explain the Relationship Between a Database and SQL	25
Describe the Purpose of DDL	28
Describe the Purpose of DML	29
Transaction Control Language	30
Build a SELECT Statement to Retrieve Data from an Oracle	
Database Table	30
Certification Summary	32
✓ Two-Minute Drill	33
Q&A Self Test	35
Self Test Answers	38
2 Using DDL Statements to Create and Manage Tables	41
Categorize the Main Database Objects	42
What Are Database Objects?	42
Schemas	44

Create a Simple Table	46
Naming a Table or Other Object	48
The SQL Statement CREATE TABLE	53
Review the Table Structure	54
List the Data Types That Are Available for Columns	56
Character	57
Numeric	58
Date	59
Large Objects	61
Explain How Constraints Are Created at the	
Time of Table Creation	63
Creating CONSTRAINTS in the	
CREATE TABLE Statement	64
The Types of CONSTRAINTS	68
Drop Columns and Set Column UNUSED	80
Dropping Columns	80
UNUSED	84
Create and Use External Tables	86
Benefits	86
Creating External Tables	87
Certification Summary	91
✓ Two-Minute Drill	94
Q&A Self Test	97
Self Test Answers	102
 3 Manipulating Data	 105
Truncate Data	106
Recursively Truncate Child Tables	107
Insert Rows into a Table	109
Default Column List	110
Enumerated Column List	113
Update Rows in a Table	117
Expressions	118
Constraints	119
The WHERE Clause	120

SELF TEST

The following questions will help you measure your understanding of the material presented in this chapter. The first five questions test you on the background information I discussed earlier to prepare you for the exam. The remaining questions begin to test you on the specific objectives of the exam itself. Furthermore, these questions are written in the style and format of the certification exam, so they can be good practice to help you get going. As is the case with the exam, some of these self test questions may have more than one correct answer, so read carefully. Choose all the correct answers for each question.

The Exam: An Overview

1. Which of the following topics are not included in the SQL Fundamentals I exam but are addressed on the SQL Associate exam? (Choose all that apply.)
 - A. MERGE
 - B. Conversion functions
 - C. FLASHBACK
 - D. External tables
2. If you focus on trying to achieve the minimum passing grade requirement for the exam, you can study more efficiently.
 - A. True
 - B. False
3. The exam is timed.
 - A. True
 - B. False
4. The 1Z0-071 exam (which is the subject of this book) has been officially validated by Oracle Corporation against which of the following versions of the Oracle database? (Choose all that apply.)
 - A. Every version
 - B. 9i
 - C. 11g
 - D. 12c

36 Chapter 1: Oracle and Structured Query Language (SQL)

5. The best exam guide you could possibly get for preparing to take and pass the 1Z0-071 certification exam, SQL Associate, is which of the following? (Choose all that apply.)
- A. This book.
 - B. The book you are holding right now.
 - C. This here book.
 - D. Don't make me tell you again.

Identify the Connection Between an ERD and a Relational Database

6. When transforming an ERD into a relational database, you often use an entity to build a database's:
- A. Table
 - B. Column
 - C. Attribute
 - D. Relationship
7. The unique identifier of a row in a database table is a(n):
- A. ID
 - B. Primary key
 - C. Primary column
 - D. Column

Explain the Relationship Between a Database and SQL

8. Which of the following is true of SQL?
- A. It is the most commonly used language for interacting with a database.
 - B. It is the only language you can use to create a database.
 - C. It is the only language you can use to interact with a database.
 - D. None of the above
9. What can you use to submit SQL statements for execution? (Choose all that apply.)
- A. PHP
 - B. Java
 - C. SQL Developer
 - D. SQL*Plus

Describe the Purpose of DDL

10. What is one of the purposes of DDL? (Choose the best answer.)
- A. Query data from a given table
 - B. Issue privileges to users

- C. Remove existing data from a database table
 - D. None of the above
- 11.** What can DDL be used for? (Choose three.)
- A. Add comments to a database table
 - B. Add columns to a database table
 - C. Add data to a database table
 - D. Add privileges for a user to a database table

Describe the Purpose of DML

- 12.** Which one of the following is a DML statement?
- A. ADD
 - B. ALTER
 - C. UPDATE
 - D. MODIFY
- 13.** Which of the following can be used to remove data from a table? (Choose two.)
- A. DELETE
 - B. UPDATE
 - C. MODIFY
 - D. ALTER

Build a SELECT Statement to Retrieve Data from an Oracle Database Table

- 14.** What can a SELECT statement be used to query? (Choose the best answer.)
- A. Only one report
 - B. Only one table
 - C. One or more reports
 - D. One or more tables
- 15.** Which of the following is *not* a capability of the SELECT statement?
- A. It can transform queried data and display the results.
 - B. It can remove data from a table.
 - C. It can join data from multiple tables.
 - D. It can aggregate database data.

SELF TEST ANSWERS

The Exam: An Overview

1. ☒ **A, B, C, and D.** See Table 1-1 for a full listing of all the topics included in both of the exams.
☒ All of the answers are correct.
2. ☒ **B.** Granted, it is a subjective issue, but Oracle Corporation specifically warns against this. One reason is that the published minimum requirement for a passing score can be changed without notice.
☒ **A** is incorrect. The statement is false.
3. ☒ **A.** Yes, the exam is timed.
☒ **B** is incorrect. The exam is timed, and it is imperative that you manage your time well when you take the exam.
4. ☒ **C and D.** The test has been officially validated against these two versions of the database and not the others.
☒ **A and B** are incorrect. The exam tests for functionality that did not exist in earlier versions of the Oracle database.
5. ☒ **A, B, C, and D.** Duh.
☒ All of the answers are correct.

Identify the Connection Between an ERD and a Relational Database

6. ☒ **A.** The table is a physical implementation of an entity.
☒ **B, C, and D** are incorrect. A column is a physical implementation of an entity's attribute. A relationship specifies how two entities relate to each other.
7. ☒ **B.** The primary key is the unique identifier in a table.
☒ **A, C, and D.** There is no particular understanding in ERDs of relational databases of an ID or a primary column. Both might be used to name a particular column or other object in the database, but they carry no particular specific meaning. A column is an attribute of an entity in a relational database.

Explain the Relationship Between a Database and SQL

8. ☒ **A.** SQL is the commonly used language for interacting with databases.
☒ **B, C, and D** are incorrect. SQL is not the only language for creating or interacting with databases, but it is the most commonly used.
9. ☒ **A, B, C, and D.** You can issue SQL statements from any of the listed options.
☒ All of the answers are correct.

Describe the Purpose of DDL

10. ☒ **B**. The GRANT statement is part of DDL and is used to issue privileges to a user.
☒ **A**, **C**, and **D** are incorrect. Querying data is performed using the DML statement SELECT. Use the DML statements UPDATE or DELETE to remove data from a database table. UPDATE can be used to remove data from one or more columns in a given row, and DELETE can be used to remove an entire row. “None of the above” does not apply since option **B** is correct.
11. ☒ **A**, **B**, and **C**. The DDL statement ALTER can be used to add columns or comments to a table. Use the DDL statement GRANT to add privileges.
☒ **C** is incorrect. There is no DDL statement to remove data from a database table. For this, use either UPDATE or DELETE. UPDATE can be used to remove data from one or more columns in a given row, and DELETE can be used to remove an entire row.

Describe the Purpose of DML

12. ☒ **C**. UPDATE is used to change data in an existing row.
☒ **A**, **B**, and **D** are incorrect. There is no ADD or MODIFY statement in SQL. ALTER is a DDL statement.
13. ☒ **A** and **B**. The DELETE statement is used to remove one or more rows from a given table. UPDATE is used to change data in one or more existing rows in a given table, and the change may include the removal of a value in a column, including the assignment of NULL to a given table’s column value for the candidate row or rows.
☒ **C** and **D** are incorrect. There is no MODIFY statement in SQL. ALTER is a DDL statement.

Build a SELECT Statement to Retrieve Data from an Oracle Database Table

14. ☒ **D**. The SELECT statement can be used to query one or more tables.
☒ **A**, **B**, and **C** are incorrect. The SELECT statement does not query reports; rather, it is used to create reports. The SELECT is not limited to querying a single table.
15. ☒ **B**. The SELECT statement is not used to remove data from a table. The act of querying merely copies data out of a table for reporting and other comparable purposes, but the SELECT statement—as a stand-alone statement—always leaves the data it encounters untouched and unmodified. However, it is possible to incorporate features of the SELECT within the context of other statements, such as INSERT and UPDATE, which are described later in this book.
☒ **A**, **C**, and **D** are incorrect. The SELECT statement is capable of transforming data, joining data, and aggregating data.

This page intentionally left blank

2

Using DDL Statements to Create and Manage Tables

CERTIFICATION OBJECTIVES

- | | | | |
|------|--|----------------|--|
| 2.01 | Categorize the Main Database Objects | 2.06 | Drop Columns and Set Columns
UNUSED |
| 2.02 | Create a Simple Table | 2.07 | Create and Use External Tables |
| 2.03 | Review the Table Structure | ✓ | Two-Minute Drill |
| 2.04 | List the Data Types That Are Available
for Columns | Q&A | Self Test |
| 2.05 | Explain How Constraints Are Created
at the Time of Table Creation | | |

T

his chapter begins to examine tables. Tables are database objects that programmers create to store the data that populates the database.

CERTIFICATION OBJECTIVE 2.01

Categorize the Main Database Objects

Database objects are the foundation of any database application. Database objects house and support everything any database application needs in order to form a working application. This section takes a high-level look at the database objects that can be created in an Oracle relational database management system (RDBMS), focusing on those objects that are tested on the exam. Here we will separate database objects into categories and discuss the relationship between objects and schemas. The rest of the book will delve into greater detail on each database object that is included in the exam.

What Are Database Objects?

You can create many different types of database objects in the Oracle RDBMS. The exam looks at eight of those objects in great detail.

The Complete List

A database consists of one or more database objects. The following list shows objects that a database developer can create in the Oracle 12c RDBMS. Those marked with an asterisk (*) are included on the exam. (Source: *SQL Language Reference Manual*.)

Clusters	Object tables
Constraints*	Object types
Contexts	Object views
Database links	Operators
Database triggers	Packages

Dimensions	<i>Restore points</i>
<i>Directories</i>	<i>Roles*</i>
<i>Editions</i>	<i>Rollback segments</i>
External procedure libraries	Sequences*
Indexes*	Stored functions/procedures
Index-organized tables	Private synonyms*
Indextypes	<i>Public synonyms*</i>
Java classes, etc.	Tables*
Materialized view logs	<i>Tablespaces</i>
Materialized views	<i>Users*</i>
Mining models	Views*

Note: Nonschema objects are italicized; all others are schema objects. See the section “Schemas” for more information.

The exam doesn’t test for all of these objects. It ignores objects such as PL/SQL program units and Java program units, as well as objects that are of more interest to database administrators. We will concern ourselves in this book only with those database objects that are included in the exam. The types of database objects on the exam are listed here, in alphabetical order:

Constraints	Synonyms
Indexes	Tables
Roles	Users
Sequences	Views

A Brief Description

Let’s take a brief look at the types of objects that are the subject of the exam:

- **TABLE** A structure that can store data. All data is stored in columns and rows. Each column’s data type is explicitly defined.
- **INDEX** An object designed to support faster searches in a table. An INDEX performs much the same way as an index to a book, by copying a relatively small, select amount of information, sorting it for speedy reference, and tying it back to locations in the table for supporting quick lookups of rows in the source table.

- **VIEW** A “filter” through which you can search a table and interact with a table but that stores no data itself and simply serves as a “window” onto one or more tables. VIEW objects can be used to mask portions of the underlying table logic for various reasons—perhaps to simplify business logic or to add a layer of security by hiding the real source of information. A VIEW can be used to display certain parts of a table while hiding other parts of the same table.
- **SEQUENCE** A counter, often used to generate unique numbers as identifiers for new rows as they are added to a table.
- **SYNONYM** An alias for another object in the database, often used to specify an alternative name for a table or view.
- **CONSTRAINT** A small bit of logic defined by you to instruct a particular table about how it will accept, modify, or reject incoming data.
- **USERS** The “owners” of database objects.
- **ROLES** A set of one or more privileges that can be granted to a user.

I will review each of these objects in greater detail throughout the book.

Each database object is considered to be either a “schema object” or a “nonschema object.” This begs the question, what is a schema?

Schemas

This section describes schemas—what they are and how they relate to database objects.

What Is a Schema?

A *schema* is a collection of certain database objects, such as tables, indexes, and views, all of which are owned by a user account. You can think of a schema as being the same thing as a user account, but there is a slight difference—the user account houses the objects owned by a user, and the schema is that set of objects housed therein. One definition of schema that you’ll often find in Oracle’s documentation (and elsewhere) is that a schema is a “logical collection of database objects.” Technically that’s true, but it depends on how logical the user chooses to be when building and placing those objects within his or her user account. Ideally there should be some sense to why all those objects are in there, and ideally a “schema” shouldn’t be just a random collection of objects. However, there is nothing

For example, the USER object is a nonschema object. A user account cannot own itself, and it cannot be owned by another user account. Therefore, the USER object, which is a user account, is a nonschema object and is a property of the database as a whole. The same is true for ROLE objects. ROLE objects represent one or more privileges that can be granted to one or more USER objects. Thus, a ROLE inherently exists at a level outside of an individual USER account—and is therefore a nonschema object. A PRIVATE SYNONYM is owned by a user account and is therefore a schema object, but a PUBLIC SYNONYM is a variation on the SYNONYM object that is owned by the special user account PUBLIC, whose owned objects are automatically available to the entire database by definition.

All other objects are schema objects such as TABLE, INDEX, VIEW, and the others listed in Table 2-1.

CERTIFICATION OBJECTIVE 2.02

Create a Simple Table

The exam expects you to be able to recognize the correct code to create a simple table. By “simple,” Oracle means that you’ll be required to define the table’s name, column names, data types, and any relevant constraints.

To create a table, we use the SQL command CREATE TABLE. The word CREATE is a SQL reserved word that can be combined with just about any database object (but not all) to form a SQL command. The syntax for the CREATE *objectType* statement is shown in this code listing:

```
CREATE objectType objectName attributes;
```

where

- *objectType* is a reference to an object listed in Table 2-1. Here are some exceptions:

You cannot create a CONSTRAINT in this way. Also, while CREATE PRIVATE SYNONYM *objectName* is correct, you use simply CREATE SYNONYM *objectName* for public synonyms, never CREATE PUBLIC SYNONYM *objectName*, which is incorrect.

- *objectName* is a name you specify according to the naming rules and guidelines described later in this chapter.
- *attributes* is anywhere from zero to a series of clauses that are unique to each individual objectType, which we'll review later.

One of the most frequent usages of the SQL command CREATE is to create a TABLE. When you create a table, you'll also create the table's columns and optionally some associated objects.

Let's look at an example of a basic CREATE TABLE statement:

```
CREATE TABLE work_schedule
  (work_schedule_id  NUMBER,
   start_date        DATE,
   end_date           DATE) ;
```

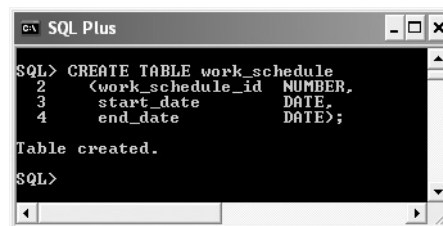
If you were to execute this command in a schema that didn't already have a table named work_schedule (that's important), then you'd get the result shown in Figure 2-1.

Let's analyze the syntax of the preceding example of a CREATE TABLE statement:

- The reserved word CREATE.
- The reserved word TABLE.
- The name of the table, chosen by you, in accordance with the rules of naming objects, which we review next.
- A pair of parentheses, in which are a series of column declarations, each separated by a comma. Column declarations consist of the following:
 - The name of the column, chosen by you, in accordance with the rules of naming objects
 - The data type of the column, taken from the list of available data types
 - A comma to separate each column definition from the next.
- A semicolon to end the statement, as is the case with all SQL statements.

FIGURE 2-1

Results of
CREATE TABLE
work_schedule
statement





TWO-MINUTE DRILL

Define Subqueries

- ☐ A subquery is a `SELECT` statement contained within a SQL statement.
- ☐ The outer SQL statement is called the parent. The outermost level is the top level.
- ☐ A top-level SQL statement containing a subquery can be a `SELECT`, `INSERT`, `UPDATE`, or `DELETE`, or else a `CREATE TABLE` or `CREATE VIEW`.
- ☐ Subqueries can be nested within other subqueries.
- ☐ Many subqueries could function as standalone queries. Some are correlated, meaning that they contain references that tie them into their parent queries.

Describe the Types of Problems Subqueries Can Solve

- ☐ Subqueries are ideal for queries based on aggregate results of other queries.
- ☐ Subqueries are used to create view objects.
- ☐ You can use a subquery in a `SELECT`, `UPDATE`, or `DELETE` statement.
- ☐ Subqueries can be used as alternatives to expressions.

Describe the Types of Subqueries

- ☐ A single-row subquery returns one row of data to the parent query.
- ☐ A multiple-row subquery may return more than one row of data to the parent query.
- ☐ Multiple-column subqueries return two or more columns' worth of data at once to the parent query, which must test for all the columns at once.
- ☐ Correlated subqueries use data from a parent query to determine their own result.
- ☐ Scalar subqueries always return one value, represented in one column of one row, every time.
- ☐ The multiple-column subquery may be of the single-row or multiple-row type of subquery.
- ☐ A correlated subquery might be a single-row, multiple-row, or multiple-column subquery.

Query Data Using Correlated Subqueries

- ☐ Correlated subqueries use data from the parent in subquery predicates to determine what data to return to the parent query.
- ☐ Correlated subqueries may present some performance degradation; however, they can perform tasks that could not otherwise be accomplished in a single query.

Update and Delete Rows Using Correlated Subqueries

- ☐ The UPDATE and DELETE statements can use correlated subqueries.
- ☐ The UPDATE statement can use correlated subqueries in the SET or WHERE clause.
- ☐ The DELETE statement can use correlated subqueries in the WHERE clause.

Use the EXISTS and NOT EXISTS Operators

- ☐ The EXISTS operator can be used by a parent query to test a subquery and determine whether it returns any rows at all.
- ☐ NOT EXISTS reverses the findings of EXISTS.

Use the WITH Clause

- ☐ The WITH clause can dynamically name a subquery so that the SELECT statement following the WITH clause can reference that subquery by name, treating it as a dynamic table in real time.
- ☐ Any subquery names assigned within the WITH clause are only good for that statement; they are not stored in the database.

Write Single-Row and Multiple-Row Subqueries

- ☐ The results of a single-row subquery can be compared to an expression within the parent using a scalar comparison operator, such as an equal sign or a greater-than or less-than sign.
- ☐ The column names are not required to match in such a comparison, but the data types must match so that the parent query can compare columns of any name to subquery columns of any name, provided the data types match.

- ☐ Multiple-row subqueries are compared differently to the parent query than single-row subqueries.
- ☐ Multiple-row subqueries are compared to expressions in the parent query by using multiple-row comparison conditions, such as IN, ANY, or ALL, in combination with single-row comparison operators such as <, >, =, !=.
- ☐ A multiple-row subquery cannot be compared to a parent query expression using a single-row comparison operator; if such an attempt is made, the statement will fail with an execution error message.

SELF TEST

The following questions will help you measure your understanding of the material presented in this chapter. Choose one correct answer for each question unless otherwise directed.

Define Subqueries

1. Which of the following forms of subquery never returns more than one row?
 - A. Scalar
 - B. Correlated
 - C. Multiple-column
 - D. None of the above

Describe the Types of Problems Subqueries Can Solve

2. Which of the following problems can be solved with a subquery? (Choose the two best answers.)
 - A. You are tasked with determining the minimum sales for every division in a multinational corporation.
 - B. You are tasked with determining which divisions in a corporation earned sales last year that were less than the average sales for all divisions in the prior year.
 - C. You are tasked with creating a view.
 - D. You are tasked with creating a sequence.

Describe the Types of Subqueries

3. Which of the following statements are true? (Choose two.)
 - A. A single-row subquery can also be a multiple-row subquery.
 - B. A single-row subquery can also be a multiple-column subquery.
 - C. A scalar subquery can also be a multiple-column subquery.
 - D. A correlated subquery can also be a single-row subquery.
4. Which subquery includes references to the parent query and thus cannot execute as a standalone query? (Choose the best answer.)
 - A. A scalar subquery
 - B. A correlated subquery
 - C. A multiple-column subquery
 - D. A referential subquery

5. Which of the following can a subquery be used in? (Choose all that apply.)
- A. An INSERT statement's SELECT
 - B. A GRANT statement
 - C. A WHERE clause in a SELECT statement
 - D. An inline view
6. An inline view is a form of a subquery.
- A. True
 - B. False

Query Data Using Correlated Subqueries

7. Review this WORK_HISTORY table.

WORK_HISTORY	
P * WORK_HISTORY_ID	NUMBER
EMPLOYEE_ID	NUMBER
START_DATE	DATE
END_DATE	DATE
SHIP_ID	NUMBER
STATUS	VARCHAR2 (10 BYTE)
PK_WORK_HISTORY	

Your task is to create a query that will list—for each ship—all of the EMPLOYEE_ID values for all the employees who have the shortest work history for their ship. In other words, if there are two ships, you want to list all the employees assigned to the first ship who have the shortest work history, all the employees assigned to the second ship who have the shortest work history, and so on. Which of the following queries will accomplish this task? (Choose two.)

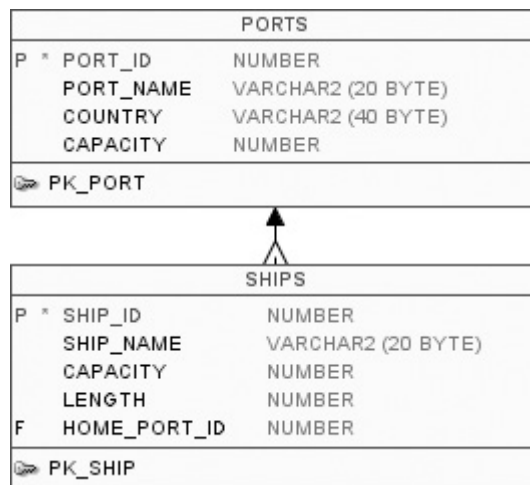
- A.

```
SELECT EMPLOYEE_ID FROM WORK_HISTORY W1
WHERE ABS (START_DATE - END_DATE) =
      (SELECT MIN (ABS (START_DATE - END_DATE))
       FROM WORK_HISTORY
       WHERE SHIP_ID = W1.SHIP_ID);
```
- B.

```
SELECT EMPLOYEE_ID FROM WORK_HISTORY W1
WHERE ABS (START_DATE - END_DATE) =
      (SELECT MIN (ABS (START_DATE - END_DATE))
       FROM WORK_HISTORY);
```

- C. `SELECT EMPLOYEE_ID FROM WORK_HISTORY W1
WHERE ABS(START_DATE - END_DATE) <= ALL
(SELECT ABS(START_DATE - END_DATE)
FROM WORK_HISTORY
WHERE SHIP_ID = W1.SHIP_ID);`
- D. `SELECT EMPLOYEE_ID FROM WORK_HISTORY W1
WHERE ABS(START_DATE - END_DATE) <
(SELECT MIN(ABS(START_DATE - END_DATE))
FROM WORK_HISTORY
WHERE SHIP_ID = W1.SHIP_ID);`

8. Review the PORTS and SHIPS tables.



Your team is tasked with the job of creating a list of the ships with the least capacity in each port. In other words, each ship has a home port. For each port that is a home port to ships, which of each port's ships has the least capacity? Your team produces the following query in answer to this task:

```

01  SELECT S1.SHIP_NAME, (SELECT PORT_NAME
02                        FROM   PORTS
03                        WHERE  PORT_ID = S1.HOME_PORT_ID) HOME_PORT
04  FROM    SHIPS S1
05  WHERE   S1.CAPACITY = (SELECT MIN(CAPACITY)
06                        FROM    SHIPS S2
07                        WHERE   S2.HOME_PORT_ID = S1.HOME_PORT_ID);

```

Which of the following statements is true about this SQL statement?

- A. The statement will fail with a syntax error because of the subquery on lines 1 through 3.
 - B. The statement will fail with an execution error because of the subquery on lines 1 through 3.
 - C. The statement will execute but will return meaningless information.
 - D. The statement will execute successfully as intended.
9. A correlated subquery:
- A. May be used in a SELECT but not an UPDATE
 - B. Cannot be executed as a standalone query
 - C. Must use a table alias when referencing a column in the outer query
 - D. All of the above

Update and Delete Rows Using Correlated Subqueries

10. Which of the following can a correlated subquery be used in? (Choose three.)
- A. The SET clause of an UPDATE statement
 - B. The WHERE clause of an UPDATE statement
 - C. The WHERE clause of a DELETE statement
 - D. The FROM clause of a DELETE statement
11. Review the illustration from question 8 and the following SQL code:

```

01 UPDATE PORTS P
02 SET   CAPACITY = CAPACITY + 1
03 WHERE EXISTS (SELECT *
04                FROM SHIPS
05                WHERE HOME_PORT_ID = P.PORT_ID);

```

The PORTS table has 15 rows. The SHIPS table has 20 rows. Each row in PORTS has a unique value for PORT_ID. Each PORT_ID value is represented in the HOME_PORT_ID column of at least one row of the SHIPS table. What can be said of this UPDATE statement?

- A. The value for CAPACITY will increase once for each of the 15 rows in the PORTS table.
- B. The value for CAPACITY will increase by 20 for each of the 15 rows in the PORTS table.
- C. The value for CAPACITY will not increase.
- D. The statement will fail to execute because of an error in the syntax.

Use the EXISTS and NOT EXISTS Operators

12. Another name for an EXISTS query is:
- A. Demijoin
 - B. Multiple-column subquery
 - C. Cross-join
 - D. Semijoin
13. Review the illustration from question 8 and the following SQL code:

```
01 DELETE FROM PORTS P
02 WHERE PORT_ID NOT EXISTS (SELECT PORT_ID
03                           FROM SHIPS
04                           WHERE HOME_PORT_ID = P.PORT_ID);
```

The code is attempting to delete any row in the PORTS table that is not a home port for any ship in the SHIPS table, as indicated by the HOME_PORT_ID column. In other words, only keep the PORTS rows that are currently the HOME_PORT_ID value for a ship in the SHIPS table; get rid of all other PORT rows. That's the intent of the SQL statement. What will result from an attempt to execute the preceding SQL statement?

- A. It will fail because of a syntax error on line 2.
- B. It will fail because of a syntax error on line 4.
- C. It will fail because of an execution error in the subquery.
- D. It will execute successfully and perform as intended.

Use the WITH Clause

14. The WITH clause can be used to name a subquery. Which of the following is also true? (Choose two.)
- A. The name of the subquery can be used in the SELECT statement following the WITH clause.
 - B. The name of the subquery can be joined to other tables in the SELECT statement following the WITH clause.
 - C. The name of the subquery is stored in the database by the WITH statement and can be referenced by other SQL statements in later sessions.
 - D. The name of the subquery can be invoked from within the subquery that is named.

15. Review the illustration from questions 8. Which of the following statements, when executed, will result in an error?
- A.

```
WITH (SELECT SHIP_ID FROM SHIPS)
SELECT PORT_ID
FROM PORTS;
```
 - B.

```
WITH SHIPPER_INFO AS
      (SELECT SHIP_ID FROM SHIPS)
SELECT PORT_ID
FROM PORTS;
```
 - C.

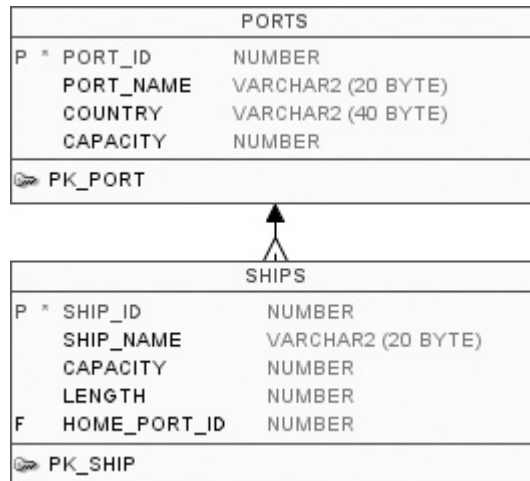
```
WITH SHIPPER_INFO AS
      (SELECT SHIP_ID FROM SHIPS)
SELECT PORT_ID
FROM PORTS, SHIPPER_INFO;
```
 - D.

```
SELECT WITH SHIPPER_INFO AS
      (SELECT SHIP_ID FROM SHIPS)
SELECT PORT_ID, SHIPPER_INFO.SHIP_ID
FROM PORTS, SHIPPER_INFO;
```

Write Single-Row and Multiple-Row Subqueries

16. Which of the following comparison operators can be used with a multiple-row subquery? (Choose two.)
- A. =
 - B. >= ALL
 - C. LIKE
 - D. IN

17. Review the PORTS and SHIPS tables:



Next, review the following SQL code:

```

01  SELECT P.COUNTRY, P.CAPACITY
02  FROM   PORTS P
03  WHERE  P.PORT_ID > (SELECT S.HOME_PORT_ID
04                      FROM SHIPS S WHERE S.LENGTH > 900);

```

You know that there are five rows in the SHIPS table with a length greater than 900. What will result from an attempt to execute this SQL statement?

- A. An execution error will result because the subquery will return more than one row and the parent query is expecting only one row from the subquery.
 - B. A syntax error will result because PORT_ID and HOME_PORT_ID in line 3 have different column names.
 - C. The statement will execute and produce output as intended.
 - D. None of the above.
- 18.** When is a query considered a multirow subquery? (Choose the best answer.)
- A. If it returns multiple rows at the time of execution
 - B. If it may or may not return multiple rows, as determined by its WHERE clause
 - C. If it returns numeric data, regardless of the number of rows of data it returns
 - D. All of the above

19. Which of the following is a true statement?
- A. If a SELECT includes a GROUP BY clause, then any subquery used within the SELECT must also have a GROUP BY clause.
 - B. If a query returns multiple rows, it may not be used as a subquery for a SELECT statement that uses a GROUP BY clause.
 - C. A SELECT statement with a GROUP BY may use a subquery to return a value to the outermost WHERE clause.
 - D. The only form of subquery permitted with a GROUP BY clause is a correlated subquery.
20. Review the PORTS and SHIPS tables shown in question 7. Then review the following SQL code:

```
01  SELECT  PORT_NAME
02  FROM    PORTS P
03  WHERE   PORT_ID IN (SELECT HOME_PORT_ID, SHIP_NAME
04                      FROM    SHIPS
05                      WHERE   SHIP_ID IN (1,2,3));
```

Which of the following is true of this statement?

- A. The statement will fail with a syntax error because of line 3.
- B. The statement will fail with a syntax error because of line 5.
- C. Whether the statement fails depends on how many rows are returned by the subquery in lines 3 through 5.
- D. None of the above.

SELF TEST ANSWERS

Define Subqueries

1. ☒ **A.** A scalar subquery always returns a single value, which is to say it returns only one column, one row.
☒ **B, C, and D** are incorrect. A correlated subquery may or may not return multiple rows. Multiple-column subqueries may or may not return multiple rows.

Describe the Types of Problems Subqueries Can Solve

2. ☒ **B and C.** The ideal query would first obtain the average sales for the prior year and then use the results to compare to the subsequent year's sales. You use a subquery to create a view.
☒ **A and D** are incorrect. The minimum sales at every division can be determined using a query with a GROUP BY. A subquery is not required or particularly necessary here, based on the information provided. You do not use a subquery to create a sequence.

Describe the Types of Subqueries

3. ☒ **B and D.** A single-row subquery may consist of multiple columns in its single row. And it also may be correlated.
☒ **A and C** are incorrect. A single-row subquery cannot, by definition, also be a multiple-row subquery. A scalar subquery by definition can be only one column in one row, so it cannot be a multiple-column subquery.
4. ☒ **B.** A correlated subquery is the best answer. The name indicates that the subquery is correlated to the parent query. Technically, scalar and multiple-column subqueries may also be correlated, but we asked for the best answer, and clearly that is correlated subquery.
☒ **A, C, and D** are incorrect. Technically, a scalar subquery may also be a correlated subquery, which is why the question asked you to pick the “best answer”—the term *correlated* refers specifically to the concept of referring to the parent query, whereas a scalar subquery refers specifically to a subquery's return value as being a single value. The same is true for a multiple-column subquery—that may also be a correlated subquery, but the term “multiple-column” is intended to emphasize the fact that it returns multiple columns' worth of results at once. Finally, a referential subquery isn't anything; we just made that up.

5. ☒ **A, C, and D.** A subquery could be used within the SELECT statement of an INSERT statement, including the SELECT statement's WHERE clause, and within an inline view.
☒ **B** is incorrect. A GRANT statement is a DCL statement and cannot have a subquery.
6. ☒ **A.** An inline view is called a view since it is a query that is treated like a table. That being said, its syntax is that of the subquery syntax.
☒ **B** is incorrect because the statement is true.

Query Data Using Correlated Subqueries

7. ☒ **A and C.** **A** is a classic correlated subquery, connecting the subquery to the parent by way of the W1.SHIP_ID value. **C** also works with the <= ALL comparison condition.
☒ **B and D** are incorrect. **B** is missing the join in the subquery that connects the subquery with the parent query. **D** compares the parent query's WHERE clause value to the subquery with a less-than sign, which won't work—the subquery is already selecting the minimum value from the subquery, so the parent query can't find anything that will be less than the minimum.
8. ☒ **D.** The statement is syntactically fine. The SELECT includes two correlated subqueries. The first, in lines 1 through 3, is an expression in the SELECT statement's select list. This subquery is correlated by way of the reference at the end of line 3. The second correlated subquery is in lines 5 through 7, and it obtains the minimum capacity value for ships belonging to each port.
☒ **A, B, and C** are incorrect. The statement will not fail, not with a syntax problem or with an execution problem. The data it returns is exactly as requested.
9. ☒ **B.** A correlated subquery cannot be executed as a stand-alone query.
☒ **A, C, and D** are incorrect. A correlated subquery can be used in a SELECT, UPDATE, or DELETE. A correlated subquery does not have to use a table alias when referencing columns unless there is a risk of ambiguity.

Update and Delete Rows Using Correlated Subqueries

10. ☒ **A, B, and C.** A correlated subquery can be used in any of these answers.
☒ **D** is incorrect. A correlated subquery cannot be used in the FROM clause of a DELETE statement. Note, however, that the question is asking specifically about correlated subqueries. While you cannot have a correlated subquery in the FROM clause of the DELETE, we'll later see that an "inline view" can be used there, and an "inline view" is essentially a subquery—but not a correlated subquery.

The Recycle Bin

The Flashback Drop feature recovers complete tables that are still retained in the “recycle bin,” and it can do so in spite of the fact that such a change results from the DROP TABLE statement, which, by definition, is a DDL statement and therefore involves an implied COMMIT. In spite of this, we can recover the table if it is still in the recycle bin.

Tables are put into the recycle bin automatically by SQL whenever a DROP TABLE statement is issued. A table’s dependent objects, such as indexes, are also placed into the recycle bin, along with the table’s constraints.

The recycle bin is not counted as space that is used by a given user account.

A user account’s dropped objects are retained in a separate recycle bin for each user. You can inspect the contents of your own recycle bin with the following query:

```
SELECT * FROM USER_RECYCLEBIN;
```

That query is identical to this one:

```
SELECT * FROM RECYCLEBIN;
```

RECYCLEBIN is a synonym for USER_RECYCLEBIN. In other words, the preceding two queries are identical.

There is a DBA_RECYCLEBIN, which allows user accounts with database administrator (DBA) privileges to see all dropped objects in the database.

If your user account has privileges on an object, then your user account will be able to see the object in the recycle bin in the event it is dropped.

You don’t need to inspect the recycle bin before issuing a Flashback statement. But you might find it helpful.

The recycle bin is affected by the “recyclebin” initialization parameter and can be turned on or off accordingly with the following ALTER SESSION statements:

```
ALTER SESSION SET recyclebin = ON;  
ALTER SESSION SET recyclebin = OFF;
```

Both of these statements take effect immediately. The initial state of the recycle bin is dependent on the setting for recyclebin in the initialization parameter file, which is controlled by the DBA.

Setting recyclebin to ON or OFF determines whether it will be used. Any objects already in the recycle bin will remain either way. To remove them, use PURGE, which is explained later in this chapter.

Dependent Objects

When a table is recovered, any associated dependent objects are also recovered, including the following:

- Indexes, except for bitmap join indexes
- Constraints, but with limitations—for example, restoring dropped tables does not recover referential constraints, meaning FOREIGN KEY constraints
- Other objects that we don't discuss in this book and that are not a subject of the exam, such as triggers

Objects that have the same name that are dropped can all be retrieved with Flashback operations. For example, if a table VENDORS was dropped and then re-created and dropped, there will be two VENDORS tables in the recycle bin. The last one dropped will be the first one retrieved.

Objects such as indexes will be recovered with system-assigned names, not the names they were originally given. You can rename each retrieved object with the RENAME TO clause of the FLASHBACK TABLE statement as it is retrieved. As of this writing, renaming retrieved objects is beyond the scope of the exam.

Statement Execution

The FLASHBACK TABLE statement operates as a single statement. If it fails, nothing in the statement succeeds. In other words, if there's an attempt to restore three tables in a single statement and the third attempt is erroneous for whatever reason, none of the tables will be restored.

PURGE

The PURGE statement permanently removes the latest version of a given item from the recycle bin—for example, to permanently remove the HOUDINI table from the recycle bin so that it cannot be recovered:

```
PURGE TABLE HOUDINI;
```

After executing this statement, the table HOUDINI cannot be recovered with the FLASHBACK TABLE statement we used earlier.

Note that the table must have first been dropped for PURGE to execute successfully. Purging may be performed automatically by the Oracle database's own automatic space reclamation operations. If that happens, the table is not in the recycle bin and cannot be recovered with Flashback operations.

You can also choose to purge the schema's recycle bin entirely.

```
PURGE RECYCLEBIN;
```

The result will be to purge all objects in the schema's recycle bin. To purge the recycle bin of the entire database, use this:

```
PURGE DBA_RECYCLEBIN;
```

The previous statement will execute successfully if the schema has the SYSDBA privilege. You can also DESCRIBE and SELECT * from the RECYCLEBIN and DBA_RECYCLEBIN.

Recovering Data Within Existing Tables over Time

This section discusses some additional ways to complete the FLASHBACK TABLE statement.

In addition to performing a Flashback operation to restore a dropped table, you can flash back an existing table to a specific point in time, showing its state prior to any committed changes that may have been transacted since the point in time of interest.

The syntax can take any of these three forms:

```
FLASHBACK TABLE HOUDINI TO SCN scn_expression;  
FLASHBACK TABLE HOUDINI TO TIMESTAMP timestamp_expression;  
FLASHBACK TABLE HOUDINI TO RESTORE POINT restore_point_expression;
```

The recommended form of these three is the first: the system change number. This is the mechanism that is recommended by Oracle for identifying points in time in the database. For example, let's revisit our table HOUDINI.

```
01 CREATE TABLE HOUDINI (VOILA VARCHAR2(30));  
02 INSERT INTO HOUDINI (VOILA) VALUES ('Now you see it.');
```

```
03 COMMIT;  
04 EXECUTE DBMS_LOCK.SLEEP(15);  
05 DELETE FROM HOUDINI;  
06 COMMIT;  
07 EXECUTE DBMS_LOCK.SLEEP(15);  
08 FLASHBACK TABLE HOUDINI TO TIMESTAMP  
09      SYSTIMESTAMP - INTERVAL '0 00:00:20' DAY TO SECOND;
```

Let's review the preceding code:

- Line 3: We commit our change to the table.
- Line 4: This is a statement that suspends processing for 15 seconds. The choice of 15 seconds is arbitrary; the intent here is to allow some time to pass.
- Line 6: We commit the deletion of one row.

- Line 8 and line 9: This statement attempts to restore the table to where it was 20 seconds earlier. Note that line 9 contains nothing more than an expression of the `TIMESTAMP` data type, which in this case is a call to the `SYSTIMESTAMP` function minus a 20-second interval. The `SYSTIMESTAMP` function returns the current time as defined by the Oracle database server's operating system. Following the call to the function is the symbol for subtraction—the minus sign—followed by a literal value representing a time interval of 20 seconds.

So, what is the result of this series of statements? Here it is:

Error report:

```
SQL Error: ORA-08189: cannot flashback the table because row movement is not enabled
08189. 00000 - "cannot flashback the table because row movement is not enabled"
*Cause:      An attempt was made to perform Flashback Table operation on a table
              for which row movement has not been enabled. Because the Flashback
              Table does not preserve the rowids, it is necessary that row
              movement be enabled on the table.
*Action:      Enable row movement on the table
```

What is wrong here? The problem is that the capability to perform Flashback operations to restore an existing table to an older state is not a capability that exists by default. It only works on tables where the `ROW MOVEMENT` feature has been enabled. Here's how to create the table with `ROW MOVEMENT` enabled:

```
CREATE TABLE HOUDINI (VOILA VARCHAR2(30))
    ENABLE ROW MOVEMENT;
```

To enable `ROW MOVEMENT` on a table that's already been created, use this:

```
ALTER TABLE HOUDINI ENABLE ROW MOVEMENT;
```

If we were to go back and redo our earlier series of statements with `ROW MOVEMENT` enabled on the table, our `FLASHBACK TABLE` statement would work perfectly, and we'd restore our table to its original state. Once it's restored, we could query the table and see all the data in that table as it existed at the time we specified in our `FLASHBACK TABLE` statement.

Data restoration is "permanent." It invokes an implicit `COMMIT` so that the restored data is committed. Note that `ROW MOVEMENT` isn't required with `flashback TO BEFORE DROP`.

Limitations

You cannot use the `FLASHBACK TABLE` statement to restore older data to an existing table if the table has been structurally altered with the `ALTER TABLE` statement in such a way that it can't accept the full definition of older data.

For example, if a column has been dropped or a column's data type changed, the FLASHBACK TABLE statement won't successfully restore the older data.

Marking Time

There are several ways to identify the point at which you want to restore data in the database. You saw three in the previous section, and I'll discuss them a bit more here.

SCN

The system change number is a numeric stamp that the database automatically increments for every committed transaction that occurs in the database. This includes both explicit and implicit commits, for all external or internal transactions. The SCN is automatically managed by the database in real time. Every committed transaction is assigned an SCN.

If you want to determine the current SCN at any given moment in the database, use the function DBMS_FLASHBACK.GET_SYSTEM_CHANGE_NUMBER. Here's an example:

```
01  SELECT DBMS_FLASHBACK.GET_SYSTEM_CHANGE_NUMBER FROM DUAL;
02
03  GET_SYSTEM_CHANGE_NUMBER
04  -----
05  5896167
```

This example shows the request and the answer. In this case, the SCN is 5896167. If you were to hesitate a few seconds and run the same statement in line 1 again, you might get a different value returned for SCN.



The SCN can also be found with the query `SELECT CURRENT_SCN FROM V$DATABASE`, which is a query of the data dictionary. We'll discuss the data dictionary in Chapter 12, but for now, note that Oracle officially recommends that if your goal is to obtain an SCN from within an application or any comparable code, then you should use the `DBMS_FLASHBACK.GET_SYSTEM_CHANGE_NUMBER` function that we demonstrated earlier. This recommendation implies that you should steer away from tapping the `V$DATABASE` view for the SCN number. The `DBMS_FLASHBACK.GET_SYSTEM_CHANGE_NUMBER` function is a function written in the PL/SQL language, and it is part of the `DBMS_FLASHBACK` package. See Oracle's reference manual on PL/SQL packages if you want to learn more—but you don't need to do that for the exam.

Each time a transaction is committed in the database, the SCN is incremented and stored with each row in each table.

The SCN for a given row can be found in the pseudocolumn `ORA_ROWSCN`. As is the case with any pseudocolumn, it can be included as an expression in any `SELECT` statement. For example, the following:

```
SELECT ORA_ROWSCN, VOILA
FROM   HOUDINI;
```

returns each row in the table `HOUDINI`, along with the values in the `VOILA` column, as well as the assigned SCN number for each row.

Timestamp

A `TIMESTAMP` value specifies a point in time. The `TIMESTAMP` data type stores the year, month, day, hour, minute, second, and fractional seconds. A literal value may be converted to the `TIMESTAMP` data type with the `TO_TIMESTAMP` function, like this:

```
SELECT TO_TIMESTAMP('2017-08-25 13:15:08.232349',
                    'RRRR-MM-DD HH24:MI:SS.FF')
FROM   DUAL;
```

This example shows the `TO_TIMESTAMP` function, a literal value, and a format mask that defines the location within the literal value of each component that forms a valid `TIMESTAMP` value. Note the use of fractional seconds, where more than two digits are accepted (up to six). Also recall that `MI` is the format mask for minutes—not `MM`, which is the format mask for months.

The `FLASHBACK_TABLE` function can use a `TIMESTAMP` value to specify a point in time in the database to within three seconds of accuracy.

If a specific point in the database is needed, don't use `TIMESTAMP`—use `SCN` instead.

If you attempt to flash back to a point in time at which the database did not exist, you'll get an error indicating “invalid timestamp specified.”

If the time you're referencing closely aligns with a time at which the object or data didn't exist, and the `SCN`/timestamp correlation misses your target and overshoots into a time frame in which the object or data didn't exist, you may get an Oracle error.

You can use a combination of one or more conversion functions to address this discrepancy, as we discuss in the next section.

Conversion Functions

SCN numbers can be converted into their equivalent `TIMESTAMP` values, and vice versa. The conversions are not exact, however, because the SCN and `TIMESTAMP` values do not represent moments in time that are precisely identical.

The conversion functions are

- **`SCN_TO_TIMESTAMP(sI)`** Takes an SCN expression as input; returns a `TIMESTAMP` value roughly corresponding to when the SCN was set
- **`TIMESTAMP_TO_SCN(t)`** Takes a `TIMESTAMP` expression representing a valid past or present timestamp as input; returns an SCN value roughly corresponding to when the `TIMESTAMP` occurred

For example, let's perform two separate conversions of timestamp values to SCN values:

```

01  SELECT  TIMESTAMP_TO_SCN(SYSTIMESTAMP) NOW,
02          TIMESTAMP_TO_SCN(TO_TIMESTAMP('01-AUG-09 09:12:23',
03                                     'DD-MON-RR HH:MI:SS')) NOT_NOW
04  FROM    DUAL;
05
06  NOW                                NOT_NOW
07  -----
08  5911139                          5639192

```

In this example, we convert the current timestamp value, as defined by `SYSTIMESTAMP`, to its SCN equivalent. We also convert a date in the past to its SCN equivalent. The first column's alias is `NOW`; the second column's alias is `NOT_NOW`.

Note that any time referenced must be within a range that is relevant to the database. In other words, the function recognizes times that apply to whenever the database installation has been in existence.

There is not a direct one-to-one relationship between timestamps and SCN values. For example, suppose you take a valid SCN value and convert it.

```

SELECT  TIMESTAMP_TO_SCN(SCN_TO_TIMESTAMP(5895585))
FROM    DUAL;

TIMESTAMP_TO_SCN(SCN_TO_TIMESTAMP(5895585))
-----
5895573

```

Note what happens here: one SCN goes in, and a slightly different SCN is ultimately returned.

For any work effort in which you require precision in specifying timing, Oracle recommends using SCN numbers and obtaining them with the DBMS_FLASHBACK packaged function GET_SYSTEM_CHANGE_NUMBER, which we reviewed earlier. Also note that if you want to use FLASHBACK to restore to a specific point, it is important that you obtain that point precisely. One way to do that is the RESTORE POINT object, which we discuss next.

RESTORE POINT

A restore point is an object in the database you create to represent a given moment in the database. That moment can be identified by a TIMESTAMP value or an SCN.

Here's an example of the CREATE RESTORE POINT statement:

```
CREATE RESTORE POINT balance_acct_01;
```

Once executed, you can use this as a restore point representing the moment at which the CREATE RESTORE POINT statement was executed. You can refer to the restore point either later in the current session or in a later session:

```
FLASHBACK TABLE HOUDINI TO RESTORE POINT balance_acct_01;
```

This statement restores the HOUDINI table to the point in time that correlates to the “balance_acct_01” restore point.

When you no longer need the RESTORE POINT, you can drop it:

```
DROP RESTORE POINT balance_acct_01;
```

You can find existing restore points with the data dictionary view V\$RESTORE_POINT. Note that users do not “own” RESTORE POINT objects; their scope is the entire database. They exist until they are dropped or age out of the control file.



New to Oracle 12c is the ability to create a multitenant cloud architecture that includes container databases and pluggable databases, but those topics are beyond the scope of the exam. It is not possible to create a restore point from within a pluggable database. But you can create a restore point in a container database. Also, remember that the execution of a DDL statement includes an implied COMMIT. CREATE RESTORE POINT is no exception. It is DDL, and therefore its execution results in a COMMIT to the database.

CERTIFICATION SUMMARY

Tables store all the data in a database. Views are named SELECT statements. Indexes act just like an index in a book; they provide a separately stored sorted summary of the column data that is being indexed, along with addressing information that points back to the source table. Sequences provide a mechanism to count off primary key values.

Views are objects that name a query and store the query in the database. Views do not store data. They look and act like tables. Views may be built with simple or complex SELECT statements. GROUP BY, HAVING, join queries, and more are all possible in a VIEW object's SELECT statement.

A SELECT statement can be used to query a view just as it would be used to query a table. But INSERT, UPDATE, and DELETE statements will work only with certain views, not all of them—it depends on whether the view's structure allows appropriate access to the table or tables upon which the view is based. Any DML statement will work with a view based on a single table, provided that the required columns are all included in the select list of the SELECT statement used to create the view, that the rows of the view are not aggregate rows, and that all of the table's constraints are honored. A view is not necessarily required to include columns that are required by the view's underlying table, but if the view omits such columns, then an INSERT statement cannot add to the view a value that is required by the underlying table; in addition, the constraints on the underlying table will reject the INSERT, and it won't work for that view.

Any SELECT statement used to create a view must have clearly defined column names. Complex expressions must be assigned a column alias; this will become the name of the column in the resulting view object.

An inline view is a variation on a subquery, in which a SELECT statement replaces a table reference in a FROM clause of any DML statement. An inline view must be enclosed in its own set of parentheses. Once incorporated in a DML statement, the inline view behaves just like a view object would behave.

If the underlying table is modified, the view may require recompilation using the ALTER VIEW . . . COMPILE statement.

A view is built with a query, and that query may specify the names of invisible columns in a table, assuming that table consists of invisible columns. Any invisible columns in the queried table that are specified in the view's query will be accessible to DML statements executed against the view.

Views can be used to hide complexity in queries. They are beneficial for creating a series of queries, one built on another. By creating a query as a view and then publishing access to that view, the developer can retain the ability to later change that query on which the view is built so that the change is reflected in the view.

Sequence objects are counters. They dispense numbers according to the parameters established when the sequence is first created.

SQL automatically defines pseudocolumns for each sequence. The `NEXTVAL` pseudocolumn increments the sequence to the next available number as defined in the sequence's parameters defined at the time the sequence is created. The `CURRVAL` pseudocolumn refers to the last `NEXTVAL` reference produced in a given session. In any session with a sequence, the `NEXTVAL` pseudocolumn must be referenced before the `CURRVAL` pseudocolumn can be referenced. A reference to a sequence pseudocolumn is a valid expression and can be invoked in many places where expressions are allowed, with certain restrictions.

An index is an object that stores a subset of presorted information from a table and may be referenced automatically by SQL to speed up the processing of a given SQL statement, depending on how that SQL statement is structured and how the Oracle Database optimizer chooses to build its execution plan. An index can be built on one or more columns in a table. Once created, if a SQL statement's `WHERE` or `ORDER BY` clause references the indexed column (or set of columns), the optimizer may use the index in its execution plan to speed up the processing of the statement. An index that is built on multiple columns is known as a *composite* index. A `UNIQUE INDEX` can be used to enforce a unique rule on a column. An index can be rendered invisible to prevent the optimizer from including the index for consideration when forming the execution plan.

Index objects support application performance tuning. Indexes can be built on one or more columns of a table; multicolumn index objects are known as composite indexes. When a query references indexed columns in its `WHERE` clause, SQL will consider using the index as part of its processing strategy, and the results may be returned more quickly.

Flashback operations can restore data to a previous point in time, with some limitations. The point to which you restore data can be identified using values of data type `TIMESTAMP`, `SCN`, or `RESTORE POINT`. Tables that have been dropped but that are still in the recycle bin can be recovered. Any table that's been purged from the recycle bin is gone for good. Data that's been changed in existing tables can be restored to some previous point in time, provided that the table's structure hasn't been changed in such a way that the table is no longer able to receive the restored data, such as perhaps having had a column dropped.



TWO-MINUTE DRILL

Describe How Schema Objects Work

- ☐ Tables store all the data in a database.
- ☐ Views are named SELECT statements.
- ☐ You can select from a view with SELECT.
- ☐ You may be able to insert, update, and delete from a view, depending on the view's structure.
- ☐ An index speeds queries on a table by creating a presorted lookup list for the table's indexed columns, along with an address pointer back to the indexed table.
- ☐ Sequences keep track of number counters to make the job of adding primary keys and other unique values easier.

Create Simple and Complex Views with Visible/Invisible Columns

- ☐ A VIEW is a SELECT statement that is stored in the database and assigned a name.
- ☐ The columns and expressions of the SELECT statement used to create a view become the columns of the VIEW.
- ☐ You can use SELECT statements on views just as you would a table.
- ☐ You can use INSERT, UPDATE, and/or DELETE statements on some views, depending on the constraints of the view's underlying table or tables, as well as other issues such as whether the view's SELECT statement includes aggregate functions.
- ☐ An inline view is a subquery that replaces a table reference in a FROM clause of a DML statement.
- ☐ The VIEW can be treated like a table, with some limitations.
- ☐ A VIEW based on a table that is subsequently altered may require recompilation with the ALTER VIEW . . . COMPILE statement.
- ☐ A VIEW can be based on a table consisting of one or more invisible columns.
- ☐ A VIEW can include a table's invisible columns in its structure by naming the invisible columns in the VIEW's query.

Create, Maintain, and Use Sequences

- ☐ A SEQUENCE is an object that dispenses numbers according to the rules established by the sequence.
- ☐ A SEQUENCE specifies the starting number, the increment (which can be positive or negative), and an optional range of values within which the sequence can generate values.
- ☐ The starting point can be within the range or at one end of the range.
- ☐ SEQUENCES are ideal for populating primary key values.
- ☐ The NEXTVAL pseudocolumn of a sequence returns the next available number in the sequence and must be the first reference to the sequence in any given login session.
- ☐ The CURRVAL pseudocolumn can return the existing value as already defined by NEXTVAL; it can be referenced in a session only after the NEXTVAL reference has occurred.
- ☐ A pseudocolumn reference to a sequence is a valid expression and can be referenced anywhere that a valid expression is allowed.
- ☐ If a valid sequence reference to NEXTVAL occurs within a DML statement that fails, the sequence still advances.

Create and Maintain Indexes Including Invisible Indexes and Multiple Indexes on the Same Columns

- ☐ An INDEX object is built on one or more columns in a table.
- ☐ The INDEX stores data from its table's columns on which it is built and presorts that data in order to speed future queries.
- ☐ When the DML statements INSERT, UPDATE, and DELETE are executed on an indexed table so that the indexed data is changed, the index is automatically updated by Oracle, thus adding to the workload of the DML statements.
- ☐ INDEX objects are built on one or more columns. Multiple-column indexes are composite indexes.
- ☐ You use the keywords VISIBLE and INVISIBLE to specify the visibility of an INDEX. The default is VISIBLE. You can specify the visibility of an index in CREATE INDEX and ALTER INDEX statements.

Perform Flashback Operations

- ☐ FLASHBACK TABLE can be used to restore a table that has been dropped.
- ☐ If a table has been dropped, it goes into the recycle bin.
- ☐ You can investigate the contents of the recycle bin to determine what objects are available for use by Flashback operations.
- ☐ Once an object has been dropped, if it is also purged with the PURGE statement, it is no longer able to be recovered with FLASHBACK TABLE.
- ☐ In addition to restoring a dropped table, the FLASHBACK TABLE statement can be used to restore data within the table as of a particular time in the database.
- ☐ Time in the database is marked by either a system change number, a RESTORE POINT, or a timestamp.

SELF TEST

The following questions will help you measure your understanding of the material presented in this chapter. Choose one correct answer for each question unless otherwise directed.

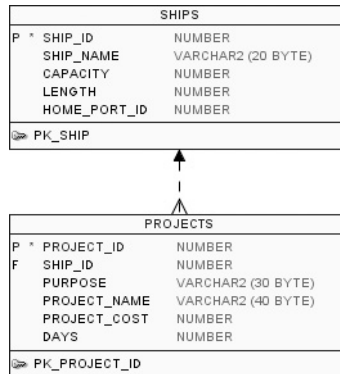
Describe How Schema Objects Work

1. The database object that stores lookup information to speed up querying in tables is:
 - A. ROWID
 - B. INDEX
 - C. VIEW
 - D. LOOKUP
2. A SEQUENCE is
 - A. Part of a table
 - B. Used exclusively to create primary key values
 - C. Is the only way to populate primary key values
 - D. None of the above.
3. All database data is stored in:
 - A. TABLES
 - B. TABLES and VIEWS
 - C. TABLES, VIEWS, and SEQUENCES
 - D. None of the above

Create Simple and Complex Views with Visible/Invisible Columns

4. Which of the following SQL statements can always be executed on any VIEW object? (Choose all that apply.)
 - A. SELECT
 - B. INSERT
 - C. DELETE
 - D. UPDATE

5. Review the following illustration:



Now review the following SQL code:

```

01 CREATE OR REPLACE VIEW SHIP_CAP_PROJ AS
02 SELECT SHIP_ID,
03         TO_CHAR(CAPACITY, '999,999'),
04         PROJECT_COST
05 FROM SHIPS JOIN PROJECTS
06 USING (SHIP_ID)
07 WHERE (PROJECT_COST * 2) < 100000;

```

What will result from an attempt to execute this SQL code?

- A. The statement will fail because of an error in line 3.
 - B. The statement will fail because of an error in line 6.
 - C. The statement will fail because of an error in line 7.
 - D. The statement will execute, and the view will be successfully created.
6. Review the illustration from question 5 and the following SQL code:

```

CREATE OR REPLACE VIEW PROJECTS_ROLLUP AS
  SELECT SHIP_NAME, CAPACITY,
         COUNT(PROJECT_ID) NUM_PROJECTS, ROUND(SUM(DAYS)) TOTAL_DAYS
  FROM SHIPS A JOIN PROJECTS B
  ON A.SHIP_ID = B.SHIP_ID
  GROUP BY SHIP_NAME, CAPACITY;

```

What can be said of this code?

- A. After the view is created, a valid SELECT statement will work on the PROJECTS_ROLLUP view, but an INSERT will not.
- B. After the view is created, a valid SELECT and valid INSERT statement will work on the PROJECTS_ROLLUP view.
- C. The attempt to create the view will fail because you cannot create a VIEW with a SELECT statement that uses a GROUP BY clause.
- D. The attempt to create the view will fail because you cannot create a VIEW with a SELECT statement that is a join.

7. Review the illustration from question 5 and the following SQL code:

```

01 CREATE OR REPLACE VIEW MAJOR_PROJECTS AS
02     SELECT PROJECT_ID, SHIP_ID, PROJECT_NAME, PROJECT_COST
03     FROM   PROJECTS
04     WHERE  PROJECT_COST > 10000;
05
06 INSERT INTO MAJOR_PROJECTS
07     (PROJECT_ID, SHIP_ID, PROJECT_NAME, PROJECT_COST)
08     VALUES
09     ((SELECT MAX(PROJECT_ID)+1 FROM PROJECTS),
10     (SELECT MAX(SHIP_ID) FROM SHIPS),
11     'Small Project',
12     500);

```

What will result from an attempt to execute these two SQL statements?

- A. The CREATE statement will fail because it omits the PURPOSE column from the PROJECTS table.
- B. The INSERT statement will fail because of an error on lines 9 and 10.
- C. The INSERT statement will fail because the PROJECT_COST value being inserted is not consistent with the WHERE clause on line 4.
- D. The CREATE and INSERT statements will successfully execute.

Create, Maintain, and Use Sequences

8. Which of the following keywords *cannot* be used with the CREATE SEQUENCE statement?
- A. CYCLE
 - B. MAXVALUE
 - C. INCREMENT
 - D. JOIN

9. Review this code:

```

DROP TABLE SHIPS CASCADE CONSTRAINTS;
DROP SEQUENCE PROJ_ID_SEQ#;
CREATE TABLE SHIPS (SHIP_ID NUMBER PRIMARY KEY,
                     LENGTH NUMBER);
CREATE SEQUENCE PROJ_ID_SEQ# START WITH 1 INCREMENT BY 4;
INSERT INTO SHIPS (SHIP_ID, LENGTH) VALUES (PROJ_ID_SEQ#.NEXTVAL, 'NOT A NUMBER');
INSERT INTO SHIPS (SHIP_ID, LENGTH) VALUES (PROJ_ID_SEQ#.NEXTVAL, 750);
COMMIT;

```

Note that the first INSERT statement is attempting to enter a string literal of 'NOT A NUMBER' into a column declared with a numeric data type. Given that, what will be the result of these SQL statements?

- A. One row added to the SHIPS table, with a SHIP_ID value of 1.
- B. One row added to the SHIPS table, with a SHIP_ID value of 5.
- C. Two rows added to the SHIPS table. The first SHIP_ID is 1; the second is 5.
- D. Two rows added to the SHIPS table. The first SHIP_ID is NULL; the second is 5.

10. Review this code:

```

DROP SEQUENCE PROJ_ID_SEQ#;
CREATE SEQUENCE PROJ_ID_SEQ# START WITH 1 INCREMENT BY 2;
SELECT PROJ_ID_SEQ#.CURRVAL FROM DUAL;

```

What will result from these SQL statements?

- A. The SELECT statement will fail because the sequence can be referenced only in an INSERT statement.
- B. The SELECT statement will fail because you cannot reference the CURRVAL pseudocolumn of a sequence until after you have referenced NEXTVAL for the sequence in a session.
- C. The SELECT statement will display a value of 1.
- D. The SELECT statement will display a value of 3.

Create and Maintain Indexes Including Invisible Indexes and Multiple Indexes on the Same Columns

11. Choose the best answer from the choices below. An index:

- A. Stores all the data from all the columns in any given table in a separate object and sorts the data for faster lookups
- B. May improve the performance of an UPDATE statement that uses a WHERE clause, if the WHERE clause performs an equality comparison on an indexed column in a table

- C. Requires a separate INSERT statement each time you add data to a table—one time to add a new row to the table, another time to add the corresponding and necessary data required by the index
- D. Only benefits a SELECT statement if the SELECT returns data that is indexed

12. Review the following series of SQL statements:

```
CREATE TABLE SUPPLIES_01
(  SUPPLY_ID NUMBER(7),
  SUPPLIER  VARCHAR2(30),
  ACCT_NO   VARCHAR2(50));
CREATE INDEX IX_SU_01 ON SUPPLIES_01(ACCT_NO);
DROP TABLE SUPPLIES_01;
CREATE TABLE SUPPLIES_02
(  SUPPLY_ID NUMBER(7),
  SUPPLIER  VARCHAR2(30),
  ACCT_NO   VARCHAR2(50));
CREATE INDEX IX_SU_02 ON SUPPLIES_02(ACCT_NO,SUPPLIER);
```

Assuming there are no objects already in existence named SUPPLIES_01 or SUPPLIES_02 prior to the execution of the preceding statements, what database objects will result from these statements?

- A. A table called SUPPLIES_02 and nothing else
 - B. A table called SUPPLIES_02 and an index called IX_SU_02
 - C. A table called SUPPLIES_02 and two indexes called IX_SU_01 and IX_SU_02
 - D. None of the above
- 13.** An invisible index is an index on one or more columns in a table:
- A. Where at least one of the columns must be invisible
 - B. Where all the columns must be invisible
 - C. And is updated for any DELETE statements performed on the table
 - D. And is updated for any SELECT statements performed on the table

Perform Flashback Operations

14. Review the following SQL code:

```
01 DROP TABLE PO_BOXES;
02 CREATE TABLE PO_BOXES (PO_BOX_ID NUMBER(3), PO_BOX_NUMBER VARCHAR2(10))
03     ENABLE ROW MOVEMENT;
04 INSERT INTO PO_BOXES VALUES (1, 'A100');
05 INSERT INTO PO_BOXES VALUES (2, 'B100');
```